

Project 2 Report

GitHub Link: https://github.com/bghosn/geog4057_badrghosn

1. Overview

This project develops a Python script and a corresponding Python toolbox that extract elevation values for water-boundary points using Google Earth Engine. The input to the tool is a CSV file containing X/Y coordinates and a flood classification raster. The tool converts the CSV to a shapefile, retrieves elevation values from the USGS 3DEP 10-meter DEM, writes the elevation values back into the shapefile, and prepares the data for visualization in ArcGIS Pro.

The final deliverables include the Python script (project2.py), the Python toolbox (Project2.pyt), the notebook used during development (Project2.ipynb), and an exported ArcGIS layout PDF.

2. Code Development

Environment Setup

A cloned ArcGIS Pro Python environment (arcpy_clone) was created to install required packages:

- earthengine-api
- geopandas
- pyogrio
- supporting dependencies for ArcPy

This environment allowed Earth Engine and GeoPandas to run alongside ArcPy.

CSV to Shapefile Conversion

The script begins by reading the CSV into a pandas DataFrame:

```
table = pd.read_csv(csv_path)
```

Using **GeoPandas**, the X and Y columns are converted into point geometry:

```
gdf = geopandas.GeoDataFrame(table)
gdf.set_geometry(geopandas.points_from_xy(gdf['X'], gdf['Y']),
```

```
inplace=True, crs=f'EPSG:{ra1.spatialReference.factoryCode}')
```

The shapefile is then written to disk using:

```
gdf.to_file(shapefile_path)
```

This creates the boundary shapefile used in ArcGIS Pro.

Adding Elevation Field

The tool automatically adds an empty **FLOAT** field named "elevation":

```
arcpy.management.AddField(shapefile_path, 'elevation', field_type='FLOAT')
```

This prepares the shapefile to receive elevation values.

Querying Google Earth Engine

The script initializes Earth Engine using the project ID:

```
ee.Initialize(project=project_name)
```

The elevation model used is:

```
dem = ee.Image('USGS/3DEP/10m')
```

Each point in the shapefile is read with a SearchCursor:

```
with arcpy.da.SearchCursor(shapefile_path, ['SHAPE@XY'], spatial_reference=sr_wgs84)
```

Each coordinate pair is converted to Earth Engine geometry and grouped into a FeatureCollection:

```
geom = ee.Geometry.Point([X, Y])  
geom_col = ee.FeatureCollection(geom_list)
```

Elevation values are sampled using:

```
elev = dem.sampleRegions(geom_col).getInfo().get('features')
```

Writing Elevation Values Back to the Shapefile

The script updates each row with the corresponding elevation value:

```
with arcpy.da.UpdateCursor(shapefile_path, ['elevation']) as cursor:  
    row[0] = elev[i]['properties']['elevation']  
    cursor.updateRow(row)
```

This finalizes the elevation-attributed point dataset that can be used inside ArcGIS Pro.

How to Use the Code

Running the Script Independently

Inside the script, paths are set in the `__main__` block:

```
csv_path = r"...boundary.csv"  
raster_path = r"...flood_2class.tif"  
shapefile_path = r"...boundary.shp"  
project_name = "ee-bghosn4"
```

Running:

```
python project2.py
```

will:

1. Convert the CSV to a shapefile
2. Add the elevation field
3. Retrieve elevation values from Earth Engine
4. Update the shapefile

Running Through ArcGIS Pro Toolbox

The .pyt toolbox exposes the script as a geoprocessing tool:

1. Add Project2.pyt to ArcGIS Pro

2. Open the tool from the Geoprocessing pane
3. Provide:
 - a. Project name
 - b. CSV path
 - c. Raster path
 - d. Output shapefile path
4. Run the tool

The result is an elevation included shapefile that can be added to ArcGIS for mapping.

Data Visualization

Once the shapefile was created, it was:

- added to a new ArcGIS map
- symbolized for clarity
- inserted into a layout
- enhanced with a **north arrow**, **scale bar**, and **legend**
- exported to PDF

This map layout is included in the GitHub repository as required.

6. Conclusion

The project demonstrates how Python, ArcPy, and Google Earth Engine can be combined to convert point coordinates into a GIS shapefile, retrieve elevation values from a remote DEM, and update spatial datasets programmatically. The resulting data was visualized in ArcGIS Pro using a formatted layout.